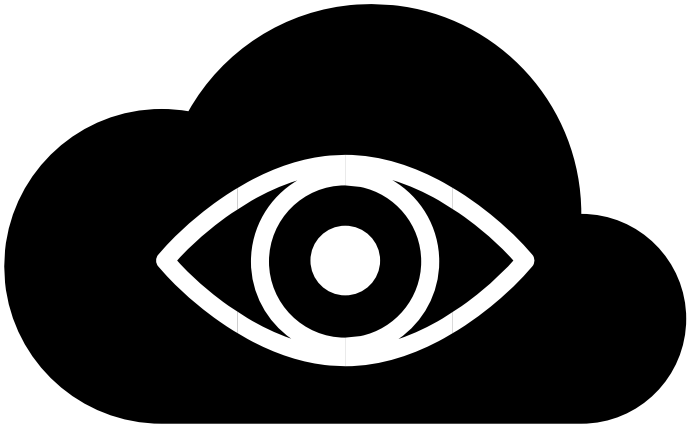


QUEST

Motoașcă Alexandru

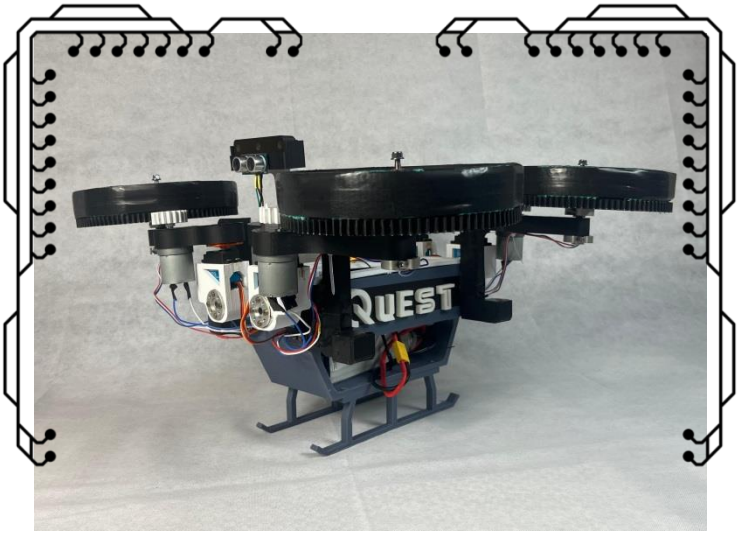
Ababei Raul

Documentație Quest

Prezentare sumară

Quest este un robot versatil, capabil să se deplaseze atât pe patru roți, cât și în zbor. Acesta poate fi controlat prin telecomandă radio sau să funcționeze autonom. În modul autonom, Quest utilizează un algoritm de inteligență artificială pentru detectarea obiectelor și ființelor, urmărind astfel persoanele.

În modul de zbor, Quest se dovedește extrem de eficient pentru monitorizarea unor zone extinse și pentru deplasarea rapidă. De exemplu, în cazul unui incendiu într-o pădure, Quest poate zbura la înălțime și poate scana rapid o suprafață mare pentru a detecta persoane sau animale aflate în pericol. Datorită camerei sale de înaltă rezoluție și a senzorilor avansați, Quest poate identifica sursele de la care izbucnește focul, permițând echipelor de intervenție să acționeze prompt și eficient.



În plus, capacitatea de zbor a lui Quest îi permite să ajungă rapid în zonele afectate, oferind o perspectivă aeriană care este esențială pentru coordonarea eforturilor de salvare și pentru evaluarea amplorii incendiului. Aceasta contribuie

la o intervenție mai rapidă și mai bine informată, reducând riscurile pentru echipele de salvare și maximizând șansele de a salva vieți și a limita pagubele. Capacitatea de a transmite imagini în timp real din aer asigură o supraveghere continuă și o adaptare rapidă la evoluția situației, fiind un instrument valoros în gestionarea situațiilor de urgență.

În modul rover, Quest nu doar că se deplasează eficient, dar poate și transporta diverse obiecte esențiale, cum ar fi bandaje, pastile sau alte echipamente medicale. Această funcționalitate este extrem de utilă în situații de urgență, unde transportul rapid și sigur al materialelor medicale poate salva vieți. De exemplu, într-un scenariu de salvare, Quest poate livra bandaje și antiseptice la locul accidentului înainte de sosirea echipelor de intervenție.

Totodată, modul rover este la fel de eficient ca modul dronă, având avantajul că poate intra mai ușor în spații închise sau dificile de accesat. Energia utilizată în modul rover este semnificativ mai mică decât în modul zbor, ceea ce îi permite să funcționeze mai mult timp pe teren.

Quest poate fi trimis în diverse misiuni la distanțe mari și transmite în timp real imaginile capturate de camera sa pe un website care poate fi accesat de oriunde. Datorită acestor funcționalități, Quest devine robotul ideal pentru situații de urgență, precum localizarea persoanelor

în locuri nesigure (persoane dispărute, victime etc.). În plus, robotul poate colecta date relevante despre mediul înconjurător, utile pentru utilizatori, contribuind astfel la evaluarea riscurilor și la luarea deciziilor rapide și informate. De exemplu, într-un scenariu de salvare a unor speologi blocați într-o peșteră, Quest poate fi trimis pentru a explora și cartografia rapid zonele inaccesibile.

În modul rover, Quest poate naviga prin tunelurile înguste și terenul accidentat al peșterii, transportând provizii esențiale precum apă, hrană sau echipamente medicale pentru cei blocați. Datorită consumului redus de energie în acest mod, Quest poate explora pe distanțe mari fără a necesita încărcare frecventă.

Pe de altă parte, în modul de zbor, Quest poate accesa rapid camerele mari sau puțurile verticale ale peșterii. Folosind senzorii săi avansați și camera de înaltă rezoluție, Quest poate detecta prezența oamenilor sau a animalelor, identifica eventuale pericole precum surse de apă sau prăbușiri de stânci și transmite în timp real imagini salvatorilor.

A. Mecanica

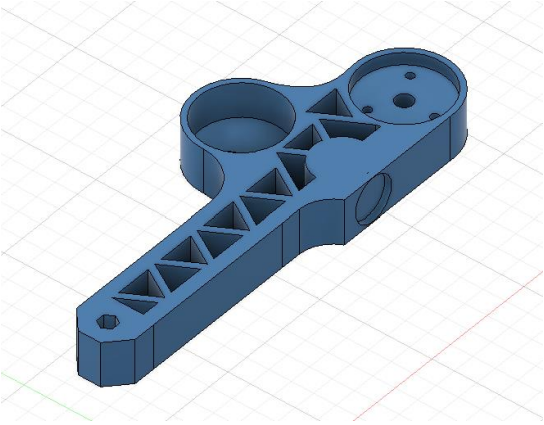
Componente:

- Raspberry Pi 4B
- Arduino Uno
- PixHawk 2.4.8
- Oak-d Lite
- 8x Motor Servo
- 4x Motor Dc
- 4x Motor Brushless
- 4in1 SpeedyBee ESC
- Modul Gps
- Senzor ultrasonic
- 2x Punte H Dubla L298n
- 3x Coborator de Tensiune

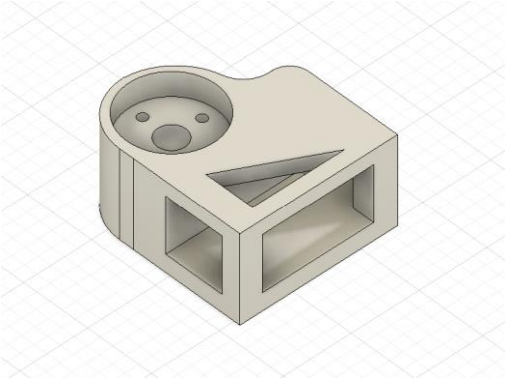
- Baterie LiPo 6s 22.2v 5000mah
- Modul Compas
- Transmitter Radio
- Receiver PPM

Materiale:

- Modele 3d printate cu: PETG-CF, PETG, PLA
- Suruburi si cuie de metal
- Capuri de servo metalice
- Magneti
- 8x Opritoare hexagonale metalice
- 4x Rulmenti hexagonali



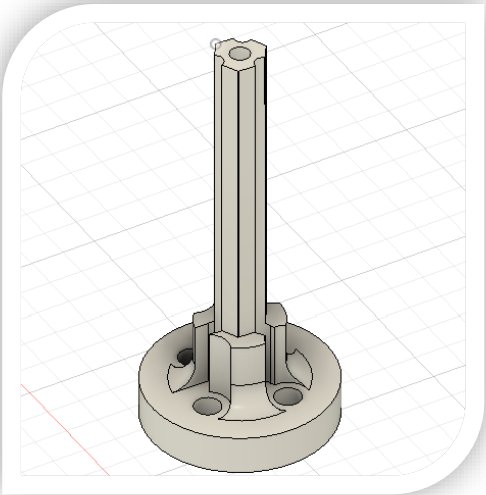
Din punct de vedere mecanic, robotul utilizează o articulație formată din două servomotoare pe fiecare braț, care permit mișcarea brațelor înainte-înapoi și sus-jos fiecare mișcare făcându-se pe o distanță de maxim 120 de grade. Aceste servomotoare permit robotului să-și ajusteze modul de deplasare, permițându-i să se întindă pentru a-și reduce înălțimea sau să se transforme în modul de dronă. Articulația robotului este formată dintr-un braț pe care sunt montate cate un motor dc motor și o roată, și o carcasă de servo ce asigură mișcarea propriu-zisă.



Brațul este printat din PETG-CF, iar carcasa din PETG normal. Ambele componente sunt gândite cu scopul de a stabiliza cât mai bine centrul de greutate al robotului, dar și de a-l nu îngreuna.

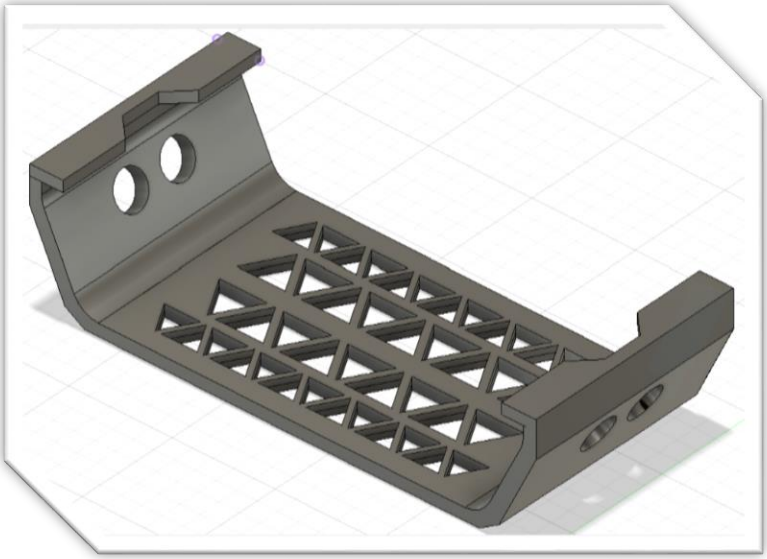
Articulația brațelor este asistată de magneti pentru bratele robotului, care reduc efortul necesar servomotoarelor și asigură stabilitatea robotului chiar și în absența curentului. O serie de magneți este poziționată pentru mersul pe patru roți la înălțimea maximă, iar cealaltă serie este poziționată pentru modul de dronă, minimizând astfel jocul servomotoarelor în timpul zborului.

Roțile robotului sunt proiectate într-un mod inovator. Motorul DC este montat la un nivel superior pe braț, în apropierea servomotoarelor, ceea ce minimizează efortul necesar mișcării brațului. Roțile sunt fixate pe rulmenți și acționează ca roți zimțate, având o bandă internă de 1 cm prevăzută cu dinți. Acești dinți se angajează precis cu roata zimțată montată pe axul motorului DC. Această configurare a permis integrarea motoarelor brushless și a elicelelor în centrul roților, permițând utilizarea roților ca suporturi pentru elice în timpul zborului. Totodată găurile din interiorul roții usureaza roata, filamentul folosit asigurând o rezistență superioară.

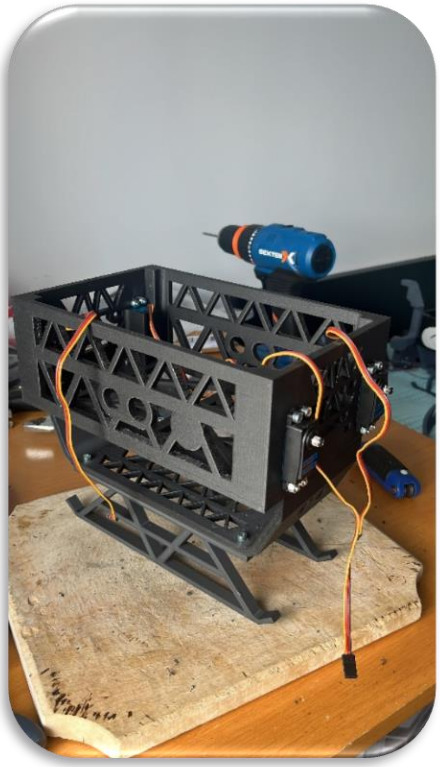


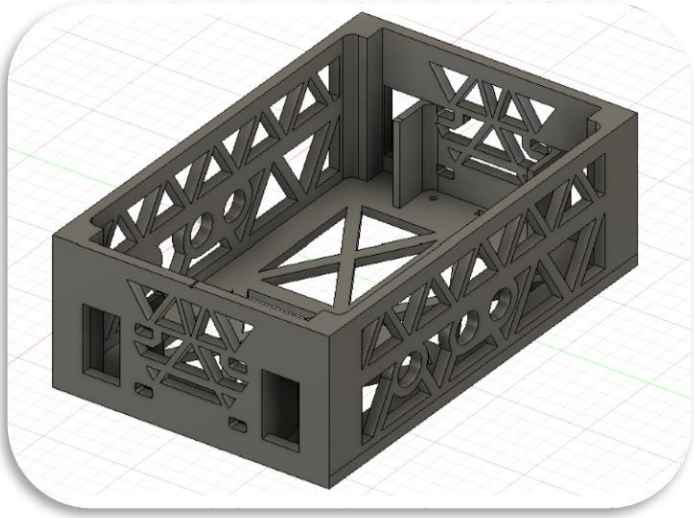
Deoarece firele motorului brushless se pot încurca în interiorul roții am dezvoltat un ax custom. Acest ax este întărit cu o tijă de metal și cele 3 fire ale motorului trec prin 3 secțiuni particulare ale axului, iar mai apoi prin gaura rulmentului roții. Astfel când roata se învârte motorul brushless rămâne staționar.

Suportul pentru baterie și carcasa robotului sunt concepute special pentru a stabiliza robotul. Bateria este amplasată exact în punctul de greutate al robotului, oferindu-i echilibru optim în timpul deplasării. În plus, această poziționare facilitează schimbarea rapidă și ușoară a bateriei. Totodată zona de sub baterie este găurită în forme de triunghiuri pentru a scapa de materialul inutil.



Cadrul robotului este conceput ergonomic, cu multiple spații libere pentru a reduce greutatea totală, fără a compromite integritatea structurală.





Stâlpii pentru magneți sunt construiți ca un accesoriu detașabil pentru body-ul robotului și ajută la menținerea măsurilor exacte ale picioarelor în cele două poziții.

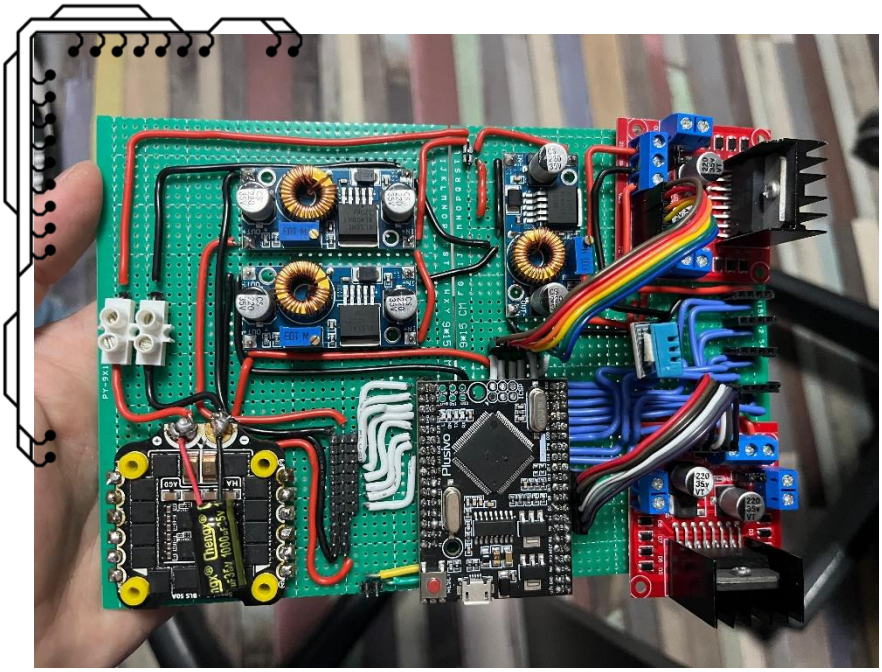
Măsuri:

- Rover (lungime 50cm, latime 39cm, inaltime 33cm)
- Drona (lungime 50cm, latime 60cm, inaltime 33cm
- Lungime maxima – 80cm
- Greutate – 4800g

B. Electronica

Din punct de vedere electronic, robotul este alimentat de la un accumulator LiPo de 22.2V 5000mah, care împarte voltajul către componentele principale cu ajutorul a trei coborâtoare de tensiune. Întregul circuit este desfășurat pe două plăci de prototipare unite, iar pe suprafața acestora se definesc patru circuite particulare: circuitul pentru motoare servo, circuitul pentru alimentarea

motoarelor DC, circuitul dedicat ESC-ului (folosit pentru controlul puterii motoarelor de zbor) și circuitul ce alimentează microprocesorul, microcontrolerul, flight computer-ul și camera inteligentă.



Corpul robotului este structurat pe două nivele. Astfel, microcontrolerul care se ocupă de mișcarea motoarelor se află pe suprafața circuitului (nivelul I), iar microprocesorul cu flight computer-ul se află pe nivelul superior, permițând accesarea ușoară și programarea rapidă a componentelor necesare.

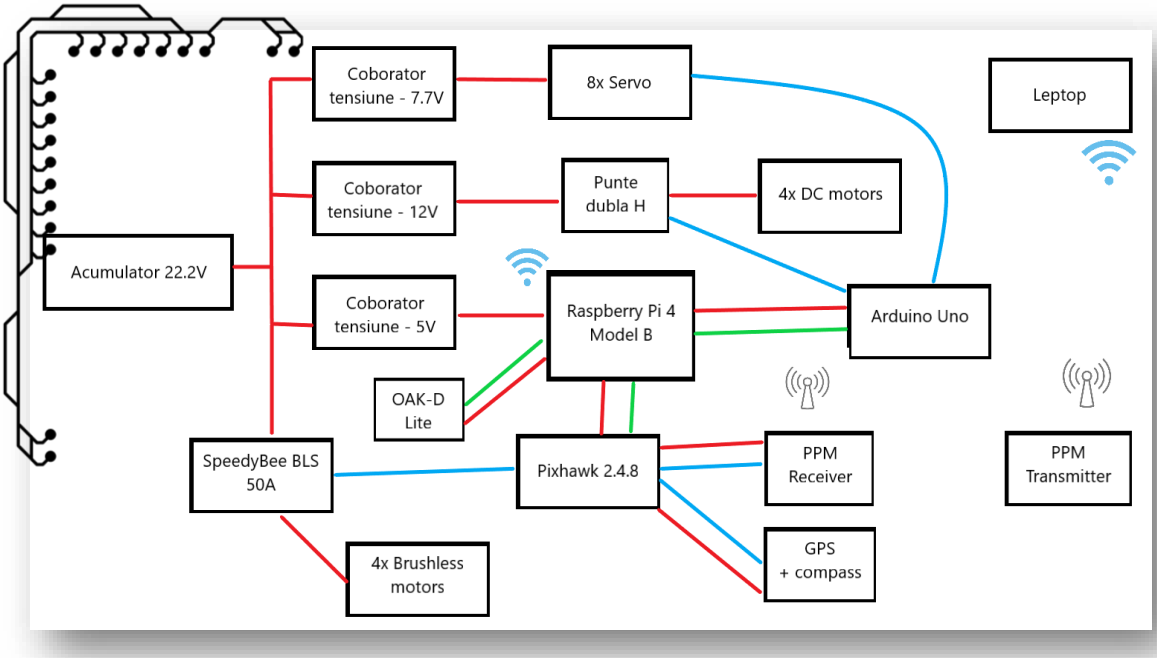
Primul circuit primește, prin coborâtorul de tensiune, voltajul de 7.7V. Pentru alimentarea celor 8 motoare servo au fost lipite 8 rânduri de pini, fiecare rând fiind dedicat unui motor, pentru a facilita ușurința folosirii componentelor. Pinii de semnal au fost conectați la plăcuța Arduino Mega care este fixată pe un pat de pini mamă și se îmbină la perfecție în pcb, asigurând astfel o conexiune eficientă și ordonată.

Al doilea circuit alimentează două punți duble L298N la voltajul de 12V. Prin intermediul acestor punți, pot fi controlate atât direcțiile motoarelor, cât și viteza acestora. Pinii de control sunt connectati, de asemenea, la plăcuța Arduino Mega, asigurând o integrare coerentă a sistemului de control. Pe lângă acest circuit, sunt

trase cablurile pentru alimentarea ESC pentru motoarele brushless. Acest ESC primește tensiunea neschimbată de 22.2V și împarte, după necesități, puterea către cele 4 motoare conectate la acesta, permițând astfel un control eficient al energiei.

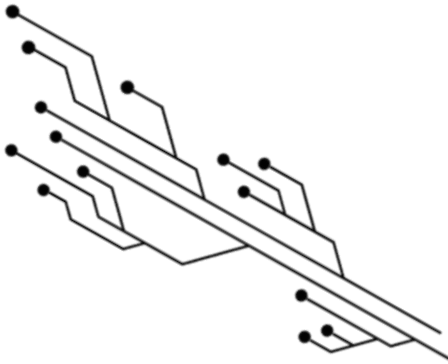
Ultimul circuit, și cel mai important, este reglat la voltajul de 5V pentru acest circuit sunt lipite 4 randuri de cate 2 pini de GND și Volți și inca 2 fire care se duc și alimentează placa Arduino Mega. La cele 4 randuri de pini sunt conectate o serie de pini care alimentează camera OAK-D Pro și microprocesorul Raspberry Pi 4 Model B prin intermediul pinilor de alimentare ai acestuia. Mai departe, microprocesorul împarte tensiunea cu ajutorul cablurilor USB Pixhawk. Astfel, comunicarea între microcontrolere si microprocesor este realizată prin conexiunea serială emulată de cablurile USB respective pini Rx, Tx. Pentru comunicarea cu UVP-ul (unitate de video procesare) camerei Oak-D Lite am folosit un USB 3.0 conenctat la Raspberry pi 4B, care ne ajută să utilizăm capabilitățile avansate ale camerei. De asemenea, microcontrolerul Pixhawk comunică cu distribuitorul de putere al motoarelor de zbor prin intermediul unui adaptor special lipit, asigurând o conexiune stabilă și sigură.

Pe lângă aceste circuite principale, sunt alimentate și transmițătorul PPM și modulul GPS + busolă. Transmițătorul PPM comunică printr-o interfață de tip semnal, GND, VCC, în timp ce modulul GPS + busolă comunică prin intermediul UART cu microcontrolerul Pixhawk. Această configurație asigură o comunicare precisă și eficientă între toate componentele robotului, permițându-i să funcționeze la parametri optimi și să execute sarcinile pentru care a fost proiectat.



(schema actuală a circuitelor robotului)

- Alimentare
- Trimitere semnal TX-RX
- Uart Serial



c. Software

1. Procesoare si controllere

Din punct de vedere software, robotul utilizează un microprocesor (Raspberry Pi 4B), o unitate de procesare video (Oak-D Pro), un microcontroler (Arduino Mega) și un calculator de zbor (Pixhawk 2.4.8). Fiecare dintre aceste componente funcționează într-un mod specific și rulează programe diferite la pornire. Comunicarea dintre toate aceste componente se realizează prin intermediul UART și conexiuni seriale, astfel încât Raspberry Pi-ul acționează ca centrul de control al întregului robot. Acesta verifică, primește și trimite date către toate celelalte

procesoare, asigurând un flux eficient de informații și coordonând funcționarea optimă a întregului sistem.

Raspberry Pi 4B, în calitate de creier al robotului, gestionează comunicarea cu celelalte componente, coordonând activitățile și colectând date de la acestea. Codul rulat pe Raspberry Pi este proiectat să fie eficient și să valorifice punctele forte ale fiecărei componente a robotului. De exemplu, unitatea de procesare video Oak-D Lite este utilizată pentru analiză video și detectare de obiecte, oferind informații esențiale pentru navigația și interacțiunile robotului.

Arduino Mega este folosit pentru controlul motoarelor servo și DC. Acesta este conectată la mai mulți senzori de distanță și temperatură. Placa Arduino trimite datele de la senzori către Raspberry Pi, dar în același timp este pregătită să primească o multitudine de date și să răspundă corespunzător acestora. Astfel, asigură o reacție rapidă la comenzile primite, în timp ce trimite constant date esențiale către website-ul robotului.

Camera Oak-D Pro se ocupă de procesarea detecției folosind modelul YOLOv8 nano, un algoritm avansat de detecție a obiectelor, și de streaming video. Aceasta capturează imagini de înaltă calitate și procesează datele în timp real, permițând robotului să recunoască și să urmărească obiecte într-un mod eficient. Integrând modelul YOLOv8n, camera poate detecta rapid și precis multiple obiecte, oferind informații critice pentru navigația și interacțiunile robotului. Funcționalitatea de streaming video permite

transmiterea fluxului video către Raspberry Pi pentru analiza suplimentară și redarea imagininilor de la distanță.

Pixhawk, echipat cu software-ul ArduPilot, gestionează calibrarea motoarelor și simplifică modul de zbor utilizând unitatea de măsurare inerțială (IMU). IMU-ul integrat colectează date despre accelerare și rotație, care sunt folosite pentru a menține stabilitatea și controlul precis al zborului. Libraria pyMavProxy automatizează procesele complexe de calibrare a motoarelor, asigurând o performanță optimă cu un efort minim din partea utilizatorului. Aceste funcționalități permit robotului să execute manevre de zbor complexe și să răspundă rapid la schimbările de mediu, folosind doar câteva funcții simplificate.

2. Cod Raspberry Pi (Backend)

```python
app = Flask(__name__)
CORS(app, supports_credentials=True)
app.secret_key = 'supersecretkey'
socketio = SocketIO(app, cors_allowed_origins="*", async_mode='eventlet')

arduino = serial.Serial('/dev/ttyS0', 115200, timeout=1)
arduino.flush()


1 usage
def connect_pixhawk():
    ports = ['/dev/ttyACM0', '/dev/ttyACM1']
    for port in ports:
        try:
            connection = mavutil.mavlink_connection(port, baud=57600)
            connection.wait_heartbeat()
            print(f"Connected to Pixhawk on {port}")
            return connection
        except Exception as e:
            print(f"Failed to connect to Pixhawk on {port}: {e}")
    raise Exception("Could not connect to Pixhawk on any known port")


pixhawk_connection = connect_pixhawk()

pixhawk_connection.wait_heartbeat()
```

Această parte a codului inițializează partea de backend scrisă în framework-ul flask care utilizează atât o instant standard cu endpointuri, cât și una de socket.io pentru redare rapidă a datelor. De asemenea, se asigură că Arduino-ul și Pixhawk-ul se conectează corect. Funcția connect_pixhawk încearcă să se conecteze la dispozitive pe porturile specificate și returnează conexiunea dacă este reușită. Dacă nu reușesc să se conecteze la niciun port, generează o excepție.

```python
pixhawk_connection.mav.request_data_stream_send(
    pixhawk_connection.target_system,
    pixhawk_connection.target_component,
    mavutil.mavlink.MAV_DATA_STREAM_RC_CHANNELS,
    5,
    1
)


1 usage
def disable_pre_arm_checks():
    print("Disabling specific pre-arm checks...")
    pixhawk_connection.mav.param_set_send(
        pixhawk_connection.target_system,
        pixhawk_connection.target_component,
        b'ARMING_CHECK',
        0,
        mavutil.mavlink.MAV_PARAM_TYPE_INT32
    )
    eventlet.sleep(2)

disable_pre_arm_checks()

1 usage
def set_stabilize_mode():
    mode_id = pixhawk_connection.mode_mapping().get('STABILIZE')
    if mode_id is None:
        print("STABILIZE mode not available")
        return False

    pixhawk_connection.mav.set_mode_send(
        pixhawk_connection.target_system,
        mavutil.mavlink.MAV_MODE_FLAG_CUSTOM_MODE_ENABLED,
        mode_id)

    start_time = time.monotonic()
    while time.monotonic() - start_time < 5:
        ack_msg = pixhawk_connection.recv_match(type='HEARTBEAT', blocking=True, timeout=1)
        if ack_msg:
            current_mode = mavutil.mode_string_v10(ack_msg)
            print(f"Current mode: {current_mode}")
            if current_mode == 'STABILIZE':
                print("STABILIZE mode set successfully")
                return True
    print("Timeout while setting STABILIZE mode")
    return False


set_stabilize_mode()
```

După stabilirea conexiunilor, sunt scrise o multitudine de funcții direcționate pixhawkului. Folosindu-ne de conexiunea formată anterior cerem să primim date de la canalele telecomenzii radio, prin interpretarea acestor canale se realizează

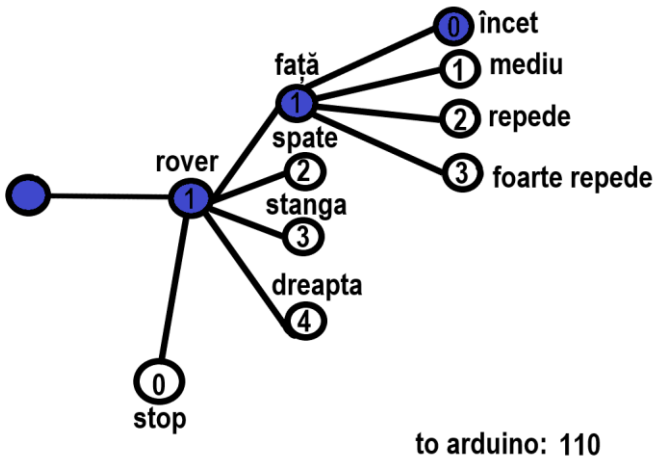
controlul robotului. Funcția disable_pre_arm_cheeks asigură că pixhawkul va fi gata să intre în modul dronă, eliminând posibilitatea ca intrarea în acest mod să cauzeze erori. Funcția set_stabilize_mode pune pixhawkul întru-un mod preprogramat, acest mod asigură că pixhawkul va asculta în mod constant la datele trimise de Raspberry Pi.

```
1 usage
def arm_command():
    print("Arming Pixhawk...")
    pixhawk_connection.mav.command_long_send(
        pixhawk_connection.target_system,
        pixhawk_connection.target_component,
        mavutil.mavlink.MAV_CMD_COMPONENT_ARM_DISARM,
        0, 1, 0, 0, 0, 0, 0, 0)


1 usage
def disarm_command():
    print("Disarming Pixhawk...")
    pixhawk_connection.mav.command_long_send(
        pixhawk_connection.target_system,
        pixhawk_connection.target_component,
        mavutil.mavlink.MAV_CMD_COMPONENT_ARM_DISARM,
        0, 0, 0, 0, 0, 0, 0, 0)


1 usage
def confirm_disarm():
    print("Confirming disarm status...")
    start_time = time.monotonic()
    while time.monotonic() - start_time < 5:  # 5 seconds timeout
        ack_msg = pixhawk_connection.recv_match(type='HEARTBEAT', blocking=True, timeout=1)
        if ack_msg:
            if ack_msg.base_mode & mavutil.mavlink.MAV_MODE_FLAG_SAFETY_ARMED == 0:
                print("Pixhawk disarmed successfully")
                return True
    print("Failed to disarm Pixhawk")
    return False


armed = False
```

Funcțiile de mai sus pun pixhawkul fie în modul armed, fie în modul disarmed. Modul armed este folosit în mod special pentru drone, acesta activează motoarele brushless și face astfel încât datele de la canalele de zbor(Ex: throttle) să coreleze cu puterea motoarelor.

```
1 usage
def read_pixhawk_data():
    global armed
    while True:
        try:
            msg = pixhawk_connection.recv_match(type='RC_CHANNELS', blocking=True)
            if msg:
                task_channel = getattr(msg, 'chan7_raw', None)
                if task_channel and 1400 <= task_channel <= 1600:
                    mode_channel = getattr(msg, 'chan8_raw', None)
                    if mode_channel is not None:
                        command = ""
                        print(f"Mode channel value: {mode_channel}")
                        if mode_channel >= 1900:
                            if not armed:
                                command = "0"
                                eventlet.sleep(2)
                                arm_command()
                                armed = True
                        elif mode_channel <= 1200:
                            if armed:
                                disarm_command()
                                eventlet.sleep(3)
                                armed = not confirm_disarm()
                            command = "1"
                            up_down_channel = getattr(msg, 'chan2_raw', None)
                            left_right_channel = getattr(msg, 'chan1_raw', None)
                            if up_down_channel is not None and left_right_channel is not None:
                                if up_down_channel <= 1475:
                                    num = (up_down_channel - 1000) / 100
                                    if num >= 0 and num <= 1:
                                        speed = "3"
                                    elif num > 1 and num <= 2:
                                        speed = "2"
                                    elif num > 2 and num <= 3:
                                        speed = "1"
                                    else:
                                        speed = "0"
```

Funcția read_pixhawk_data este primul nostru thread, acesta asigură mișcarea robotului și comunicarea între pixhawk și arduino. Se interpretează datele de la canalele radio, iar pe urmă se trimit către Arduino. Fapt ce dă viață robotului nostru. Exemplu comandă trimisă către Arduino:



```python
def get_depthai_pipeline():
    nnPath = blobconverter.from_zoo(name="yolov8n_coco_640x352", zoo_type="depthai", shaves=6)

    pipeline = dai.Pipeline()

    camRgb = pipeline.create(dai.node.ColorCamera)
    detectionNetwork = pipeline.create(dai.node.YoloDetectionNetwork)
    xoutRgb = pipeline.create(dai.node.XLinkOut)
    nnOut = pipeline.create(dai.node.XLinkOut)

    xoutRgb.setStreamName("rgb")
    nnOut.setStreamName("nn")

    camRgb.setPreviewSize(640, 352)
    camRgb.setResolution(dai.ColorCameraProperties.SensorResolution.THE_1080_P)
    camRgb.setInterleaved(False)
    camRgb.setColorOrder(dai.ColorCameraProperties.ColorOrder.BGR)
    camRgb.setFps(15)

    detectionNetwork.setConfidenceThreshold(0.5)
    detectionNetwork.setNumClasses(80)
    detectionNetwork.setCoordinateSize(4)
    detectionNetwork.setAnchors([10, 14, 23, 27, 37, 58, 81, 82, 135, 169, 344, 319])
    detectionNetwork.setAnchorMasks({"side26": [1, 2, 3], "side13": [3, 4, 5]})
    detectionNetwork.setIouThreshold(0.5)
    detectionNetwork.setBlobPath(nnPath)
    detectionNetwork.setNumInferenceThreads(2)
    detectionNetwork.input.setBlocking(False)

    camRgb.preview.link(detectionNetwork.input)
    detectionNetwork.passthrough.link(xoutRgb.input)
    detectionNetwork.out.link(nnOut.input)

    return pipeline


pipeline = get_depthai_pipeline()
device = dai.Device(pipeline)
rgb_queue = device.getOutputQueue(name="rgb", maxSize=4, blocking=False)
nn_queue = device.getOutputQueue(name="nn", maxSize=4, blocking=False)
```

Se creează un pipeline, folosind DepthAI pentru camera Oak-D Pro. Prin acest pipeline ne vom asigura că procesarea este făcută pe videoprocesorul camerei. Se definesc sursele și ieșirile, se configurează proprietățile camerei și ale rețelei YOLO, și se setează parametrii specifici pentru detectarea obiectelor. Camera RGB capturează imaginile la o rezoluție de 640x352, optimă pentru modelul YOLO. Se setează un prag de confidence de 50% și se configurează parametrii specifici YOLO, cum ar fi ancorele, cele 80 de clase pentru fiecare tip de obiect detectat și măștile de ancore. Modelul YOLO este încărcat din locația specificată de blob_path.

```python
@socketio.on('connect')
def handle_connect():
    print('Client connected')
    emit('hello', 'world')


@socketio.on('disconnect')
def handle_disconnect():
    print('Client disconnected')


1 usage
def generate_frames():
    while True:
        try:
            inRgb = rgb_queue.get()
            inDet = nn_queue.get()

            if inRgb is not None:
                frame = inRgb.getCvFrame()
                if inDet is not None:
                    detections = inDet.detections
                    frame = displayFrame(frame, detections)

                encode_param = [int(cv2.IMWRITE_JPEG_QUALITY), 30]
                ret, buffer = cv2.imencode('.jpg', frame, encode_param)
                frame = base64.b64encode(buffer).decode('utf-8')

                socketio.emit('video_frame', {'frame': frame})
                eventlet.sleep(0.075)
        except Exception as e:
            print(f"Error in generate_frames: {e}")
            eventlet.sleep(1)
```

În aceasta parte de cod, se creează conexiunea între client și server prin socket.io, dar și funcția de disconnect. Funcția generate_frames preia videoul redat de cameră, pentru fiecare frame se compresează imaginea pentru o redare cât mai rapidă, iar apoi fiecare frame se trimite prin socket.io.emit. Funcția socket.io.emit trimite prin intermediul a websockets care sunt mai rapide decât un endpoint normal fiecare frame encodat. Frameurile sunt redate la un interval de 75 milisecunde.

```python
@app.route('/login', methods=['POST'])
def login():
    data = request.json
    username = data.get('username')
    password = data.get('password')

    if username == 'quest' and password == 'iosub':
        session['logged_in'] = True
        print("Login successful")
        return jsonify({'success': True}), 200
    else:
        print("Login failed")
        return jsonify({'success': False}), 401


@app.route('/sensor_data')
def sensor_data():
    data = read_serial_data()
    if data:
        sensor_data = data.split(',')
        if len(sensor_data) != 6:
            return jsonify({'error': 'Invalid sensor data format'}), 500
        try:
            front_distance = int(sensor_data[0])
            right_distance = int(sensor_data[1])
            left_distance = int(sensor_data[2])
            back_distance = int(sensor_data[3])
            temperature = float(sensor_data[4])
            humidity = float(sensor_data[5])

            return jsonify({
                'front': front_distance,
                'right': right_distance,
                'left': left_distance,
                'back': back_distance,
                'temperature': temperature,
                'humidity': humidity
            })
        except ValueError as e:
            return jsonify({'error': f'Invalid sensor data values: {e}'}), 500

    return jsonify({
        'error': 'No sensor data received from Arduino'
    }), 500
```

Endpointul de login asigură că utilizatorul este logat cu datele corecte înainte de a vedea videoul trimis de cameră și datele de la senzori. Am înlocuit abordarea standard cu salvarea utilizatorului într-o bază de date cu una mai simplă. Utilizatorul are o parolă și un nume predefinit fără baza de date, iar pentru a verifica dacă datele sunt corecte se verifica doar dacă șirurile de caractere coincid

între ele întrucât exista un singur utilizator în toată aplicația momentan. Endpointul /sensor_data trimite în format jason datele de la senzorii de distanță și de temperatură.

```python
if __name__ == "__main__":
    eventlet.spawn(read_pixhawk_data)
    eventlet.spawn(generate_frames)
    socketio.run(app, host='0.0.0.0', port=5000)
```

Funcția main pornește backend-ul, folosindu-se de eventlet se pornesc ambele threaduri și conexiunea socket.io pe portul 5000.

3. Frontend

```javascript
const socket = io('https://quest-robot-app.eu.ngrok.io', {
  transports: ['websocket'],
  reconnectionAttempts: 5,
  reconnectionDelay: 1000,
  timeout: 20000,
});

function VideoStream() {
  const [frame, setFrame] = useState(null);

  useEffect(() => {
    socket.on('connect', () => {
      console.log('Connected to WebSocket server');
    });

    socket.on('hello', (arg) => {
      console.log('Received hello:', arg);
    });

    socket.on('video_frame', (data) => {
      console.log('Received video frame');
      const imageSrc = ${data.frame};
      setFrame(imageSrc);
    });

    socket.on('disconnect', (reason) => {
      console.log('Disconnected from WebSocket server:', reason);
    });

    socket.on('reconnect_attempt', () => {
      console.log('Reconnecting...');
    });

    socket.on('reconnect_failed', () => {
      console.log('Reconnection failed');
    });

    return () => {
      socket.off('hello');
      socket.off('video_frame');
      socket.off('connect');
      socket.off('disconnect');
      socket.off('reconnect_attempt');
      socket.off('reconnect_failed');
    };
  }, []);
```

Fișierul de cod videoStream.js apelează prin socket.io linkul public al backendului și primește în mod constant imaginile encodate redate de cameră pe care le transformă în imagini vizibile.

```
const handleLogin = async (e) => {
  e.preventDefault();
  try {
    const response = await axios.post('https://quest-robot-app.eu.ngrok.io/login', {
      username,
      password,
    }, { withCredentials: true });
    if (response.data.success) {
      sessionStorage.setItem('loggedIn', 'true');
      navigate('/video');
    } else {
      setError('Invalid credentials');
    }
  } catch (error) {
    setError('Error logging in');
  }
};

return (
  <div className="App">
    <header className="App-header">
      <h1>Login</h1>
      <form onSubmit={handleLogin}>
        <input
          type="text"
          placeholder="Username"
          value={username}
          onChange={(e) => setUsername(e.target.value)}
        />
        <input
          type="password"
          placeholder="Password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
        />
        <button type="submit">Login</button>
      </form>
      {error && <p style={{ color: 'red' }}>{error}</p>}
    </header>
  </div>
);
;
```

În această parte de cod se face autentificarea utilizatorului, apelăm endpointul /login și dacă autentificarea a fost cu succes redirecționăm către pagină cu sufixul /video pentru a putea vedea videoul. Ne folosim de librăria axios pentru a

securiza paginile și pentru a nu permite ca videoul să fie accesat fără credențialele corecte.

4. Hosting (acces la site de pe orice dispozitiv cu conexiune la internet)

Pentru a permite accesul la site și imagine de oriunde, am folosit ngrok pentru a expune serverul local de pe portul 5000 printr-un tunel, oferindu-i o adresă publică. Frontend-ul, scris în React, apelează endpointurile la această adresă publică și este găzduit pe Firebase.

5. Cod Arduino

```
bool ok = true;
void loop() {
  if(ok==true)
  {
    Drive();
    ok= false;
  }
  int front_distance = front_sensor.ping_cm();
  int right_distance = right_sensor.ping_cm();
  int left_distance = left_sensor.ping_cm();
  int back_distance = back_sensor.ping_cm();

  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();

  Serial.print(front_distance);
  Serial.print(",");
  Serial.print(right_distance);
  Serial.print(",");
  Serial.print(left_distance);
  Serial.print(",");
  Serial.print(back_distance);
  Serial.print(",");
  Serial.print(temperature);
  Serial.print(",");
  Serial.println(humidity);
  if (Serial.available() > 0) {
        String incomingString = Serial.readStringUntil('\n');

        char mode = incomingString[0];

        if (mode == '0') {
            // Fly command
            Drone();
        } else if (mode == '1') {
            int direction = incomingString[1] - '0';
            int power = incomingString[2] - '0';
            // Drive command
            Drive();

            driveWheels(direction, power);
        }
    }
}
```

Mai întâi se afișează distanțele de la senzorii ultrasonici și se citește temperatura și umiditatea de la senzorul dht. Datele de la senzorii sunt trimise prin serial către arduino separate prin “,”. Comenzile trimise de către Raspberry Pi sunt preluate de Arduino și folosite pentru a transmite motoarelor nevoia pe care

o cere Raspberry. În loop-ul principal se face conectarea serială dintre cele două componente. Se citește string-ul transmis de Raspberry și dacă acest string nu este NULL, se începe decodificarea acestuia.

În primul rând, se stabilește modul (ZBOR/ROVER). În cazul în care modul este ROVER decodificarea continuă cu direcția și puterea motoarelor DC. Pentru toate aceste taskuri sunt pregătite funcții prestabilite. În cazul în care conexiunea serială nu este stabilită sau string-ul transmis de către Raspberry nu este corect, Quest intră în modul ROVER pentru siguranță.

```
void Drone(){
    int currentAnglesSwitch[4] = {switchServos[0].read(), switchServos[1].read(), switchServos[2].read(), switchServos[3].read()};
    int anglesSwitch[4] = {115, 69, 177, 80};
    int anglesSwitchReached[4] = {0};
    if(currentAnglesSwitch[0] != anglesSwitch[0]){
      driveWheels(5, 0);
      adjustLegsStretched();
      driveWheels(0, -1);
      delay(300);
      int currentAnglesLegs[4] = {legServos[0].read(), legServos[1].read(), legServos[2].read(), legServos[3].read()};
      int anglesLegs[4] = {108, 52, 148, 60};
      int anglesLegsReached[4] = {0};
      while (anglesSwitchReached[0] + anglesSwitchReached[1] + anglesSwitchReached[2] + anglesSwitchReached[3] != 4 &&
      anglesLegsReached[0] + anglesLegsReached[1] + anglesLegsReached[2] + anglesLegsReached[3] != 4) {
          for (int i = 0; i < 4; i++) {
              if (currentAnglesSwitch[i] != anglesSwitch[i]) {
                  int increment = (currentAnglesSwitch[i] < anglesSwitch[i]) ? 1 : -1;
                  currentAnglesSwitch[i] += increment;
                  switchServos[i].write(currentAnglesSwitch[i]);
              } else {
                  anglesSwitchReached[i] = 1;
              }
              if (currentAnglesLegs[i] != anglesLegs[i]) {
                  int increment = (currentAnglesLegs[i] < anglesLegs[i]) ? 1 : -1;
                  currentAnglesLegs[i] += increment;
                  legServos[i].write(currentAnglesLegs[i]);
              } else {
                  anglesLegsReached[i] = 1;
              }
          }
          delay(10);
      }
    }
}
```

Funcțiile Drive() și Drone() sunt funcțiile ce mișcă servourile în pozițiile de zbor și mers. Aceste două funcții sunt inversabile, adică respectă aceeași pași de mișcare, doar că invers. Spre exemplu vom lua funcția Drone().

```
void Drive(){
    int currentAnglesSwitch[4] = {switchServos[0].read(), switchServos[1].read(), switchServos[2].read(), switchServos[3].read()};
    int anglesSwitch[4] = {16, 172, 73, 180};
    int anglesSwitchReached[4] = {0};
    if(currentAnglesSwitch[0] != anglesSwitch[0]){
      int currentAnglesLegs[4] = {legServos[0].read(), legServos[1].read(), legServos[2].read(), legServos[3].read()};
      int anglesLegs[4] = {8, 157, 48, 160};
      int anglesLegsReached[4] = {0};
      while (anglesSwitchReached[0] + anglesSwitchReached[1] + anglesSwitchReached[2] + anglesSwitchReached[3] != 4 &&
      anglesLegsReached[0] + anglesLegsReached[1] + anglesLegsReached[2] + anglesLegsReached[3] != 4) {
          for (int i = 0; i < 4; i++) {
              if (currentAnglesSwitch[i] != anglesSwitch[i]) {
                  int increment = (currentAnglesSwitch[i] < anglesSwitch[i]) ? 1 : -1;
                  currentAnglesSwitch[i] += increment;
                  switchServos[i].write(currentAnglesSwitch[i]);
              } else {
                  anglesSwitchReached[i] = 1;
              }

              if (currentAnglesLegs[i] != anglesLegs[i]) {
                  int increment = (currentAnglesLegs[i] < anglesLegs[i]) ? 1 : -1;
                  currentAnglesLegs[i] += increment;
                  legServos[i].write(currentAnglesLegs[i]);
              } else {
                  anglesLegsReached[i] = 1;
              }
          }
          delay(10);
      }
      delay(300);
      driveWheels(6, 0);
      adjustLegsNormal();
      driveWheels(0, -1);
    }
```

În primul rând se citesc pozițiile actuale ale servourilor sus-jos și pozițiile lor viitoare, se verifică dacă acestea nu sunt deja în poziția bună, iar dacă ele nu sunt, robotul intră în funcția de stretch.

Această funcție secundară întinde picioarele prin servourile față-spate, totodată activându-se motoarele DC ce ușurează mișcarea picioarelor prin învârtire pe suprafața terenului. După aceasta întindere, se iau angle-urile actuale ale servourilor față-spate și pozițiile viitoare și se incepe o mișcare coordonată prin intermediul principiului inverse kinematics pentru a se ajunge la poziția de zbor. Mișcarea este simultană (toate servourile se mișcă în același timp), astfel facilitând o ridicare concisă, fără a se lovi brațele de stâlpii magnetici sau sania dronei.

```
switch (direction) {
    case 1:
        // Forward
        for (int i = 0; i < 4; i++) {
            digitalWrite(motorPins[i][0], HIGH);
            digitalWrite(motorPins[i][1], LOW);
        }
        break;
    case 2:
        // Backward
        for (int i = 0; i < 4; i++) {
            digitalWrite(motorPins[i][0], LOW);
            digitalWrite(motorPins[i][1], HIGH);
        }
        break;
    case 3:
        // Counterclockwise
        for (int i = 0; i < 3; i += 2) {
            digitalWrite(motorPins[i][0], LOW);
            digitalWrite(motorPins[i][1], HIGH);
        }
        for (int i = 1; i < 4; i += 2) {
            digitalWrite(motorPins[i][0], HIGH);
            digitalWrite(motorPins[i][1], LOW);
        }
        break;
    case 4:
        // Clockwise
        for (int i = 0; i < 3; i += 2) {
            digitalWrite(motorPins[i][0], HIGH);
            digitalWrite(motorPins[i][1], LOW);
        }
        for (int i = 1; i < 4; i += 2) {
            digitalWrite(motorPins[i][0], LOW);
            digitalWrite(motorPins[i][1], HIGH);
        }
        break;
    case 5:
        //Strech
        for (int i = 0; i < 2; i++) {
            digitalWrite(motorPins[i][0], HIGH);
            digitalWrite(motorPins[i][1], LOW);
        }
        for (int i = 2; i < 4; i++) {
            digitalWrite(motorPins[i][0], LOW);
            digitalWrite(motorPins[i][1], HIGH);
        }
        break;
    case 6:
        //Shrink
        for (int i = 0; i < 2; i++) {
            digitalWrite(motorPins[i][0], LOW);
            digitalWrite(motorPins[i][1], HIGH);
        }
        for (int i = 2; i < 4; i++) {
            digitalWrite(motorPins[i][0], HIGH);
            digitalWrite(motorPins[i][1], LOW);
        }
        break;
    case 0:
        for (int i = 0; i < 4; i++) {
            digitalWrite(motorPins[i][0], LOW);
            digitalWrite(motorPins[i][1], LOW);
        }
        break;
}
```

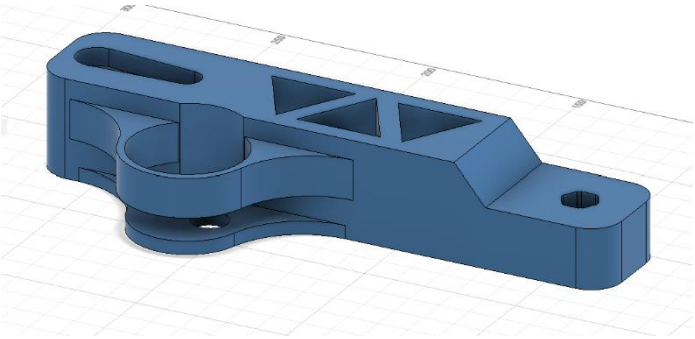
Funcția Drive() este inversă funcției Drone(), adică mai întâi se efectuează mișcarea coordonată, iar apoi retragerea picioarelor care, la fel, sunt ajutate de roți ce se învârt în sens opus stretching-ului pentru a ajuta ridicare robotului.

```
void driveWheels(int direction, int power) {
    // direction: 0=Stop, 1=Forward, 2=Backward, 3=Counter
    switch(power) {
        case 0:
            for (int i = 0; i < 2; i++) {
                analogWrite(motorEnPins[i], 100);
            }
            break;
        case 1:
            for (int i = 0; i < 2; i++) {
                analogWrite(motorEnPins[i], 150);
            }
            break;
        case 2:
            for (int i = 0; i < 2; i++) {
                analogWrite(motorEnPins[i], 200);
            }
            break;
        case 3:
            for (int i = 0; i < 2; i++) {
                analogWrite(motorEnPins[i], 255);
            }
            break;
        case -1:
            for (int i = 0; i < 2; i++) {
                analogWrite(motorEnPins[i], 0);
            }
            break;
    }
```

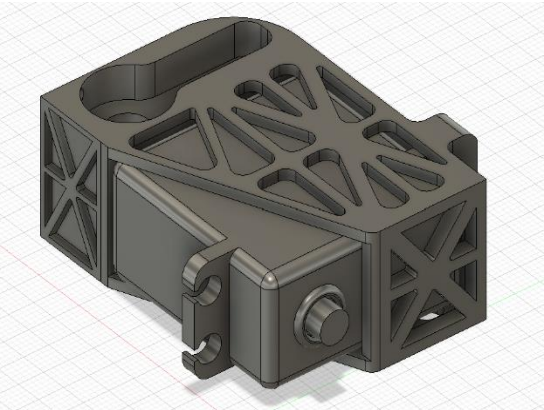
Funcția driveWheels() este folosită pentru a da direcție și putere motoarelor DC. Există două switch-uri, una setează puterea, iar cealaltă direcția. Sunt 4 niveluri de putere și 6 moduri direcții specifice observabil comentate în snippet-urile de mai jos.

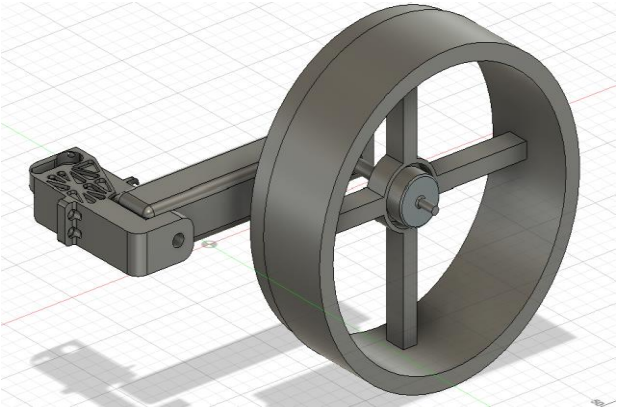
d. Parcusul robotului

Robotul a trecut prin mai multe versiuni până să ia forma actuală. Data începerii proiectului este 15 februarie 2024 și marchează ziua 0 a prietenului nostru Quest. Inițial robotul a fost gândit într-o variantă redusă în mărimi, dar care pe parcurs a ajuns să ia amploare în mărimi si greutate. Robotul a trecut de asemenea printr-o etapă lungă de gândire și regândire a sistemelor, problema principală fiind carcasele de servo și brațele roților.

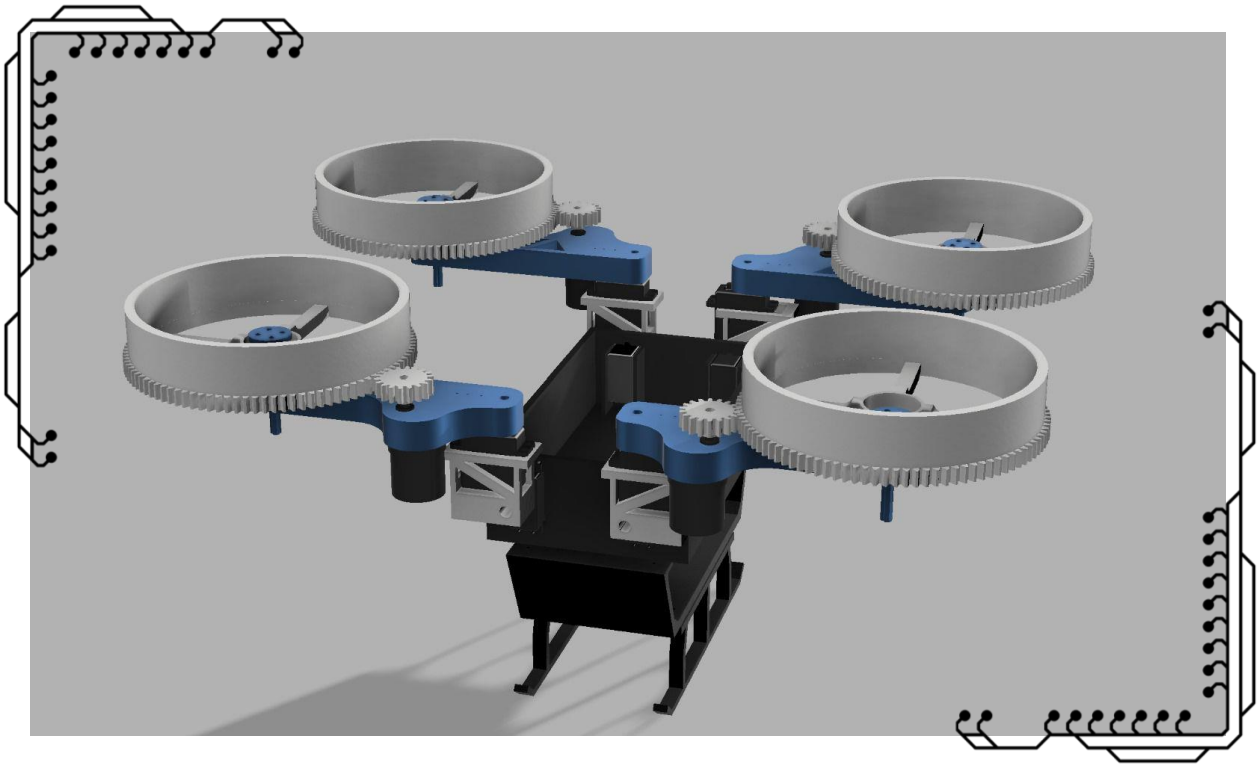


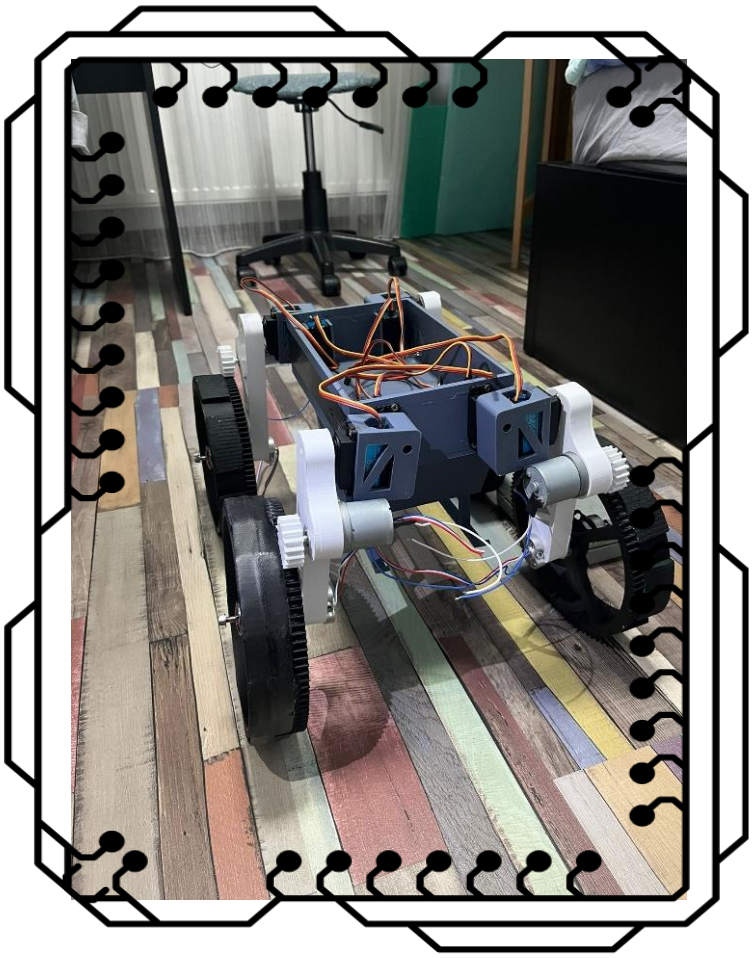
Variantele inițiale ale modelelor 3D ne-au ajutat să gândim robotul mai în detaliu și să ne aprofundăm tot mai tare în domeniul roboticii. Zi de zi au avut loc căutări de idei tot mai bune și mai sigure, arătându-ne tot mai tare o imagine de ansamblu a ceea ce avea să ajungă ideea noastră.



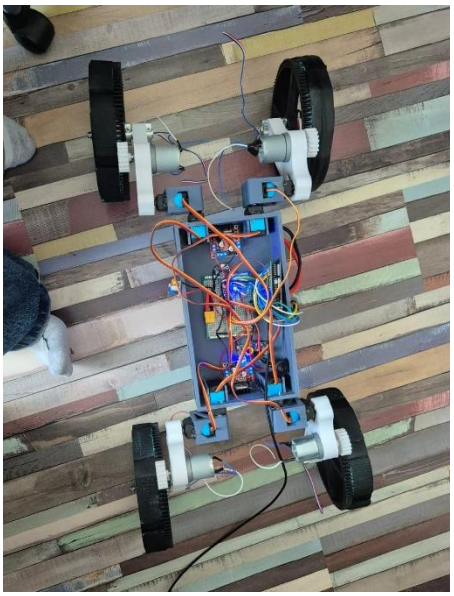


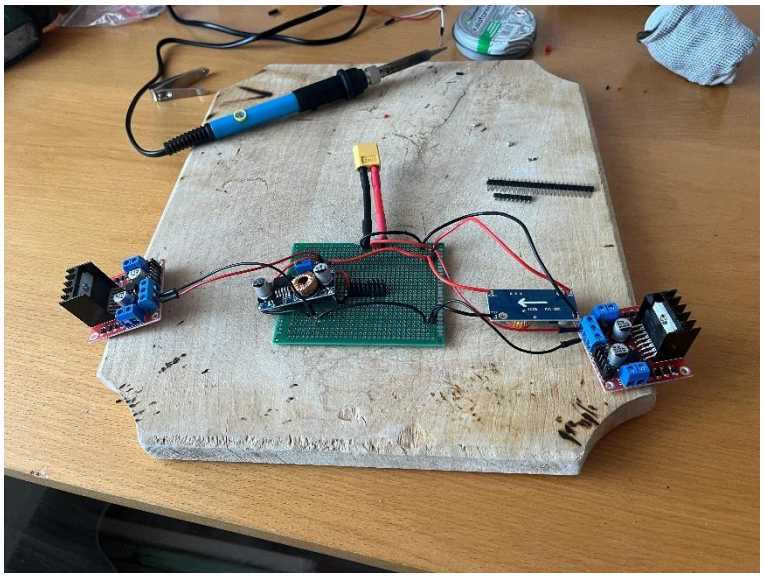
Printurile au fost schimbate, iar prima iterație a robotului a ajuns să fie văzută:





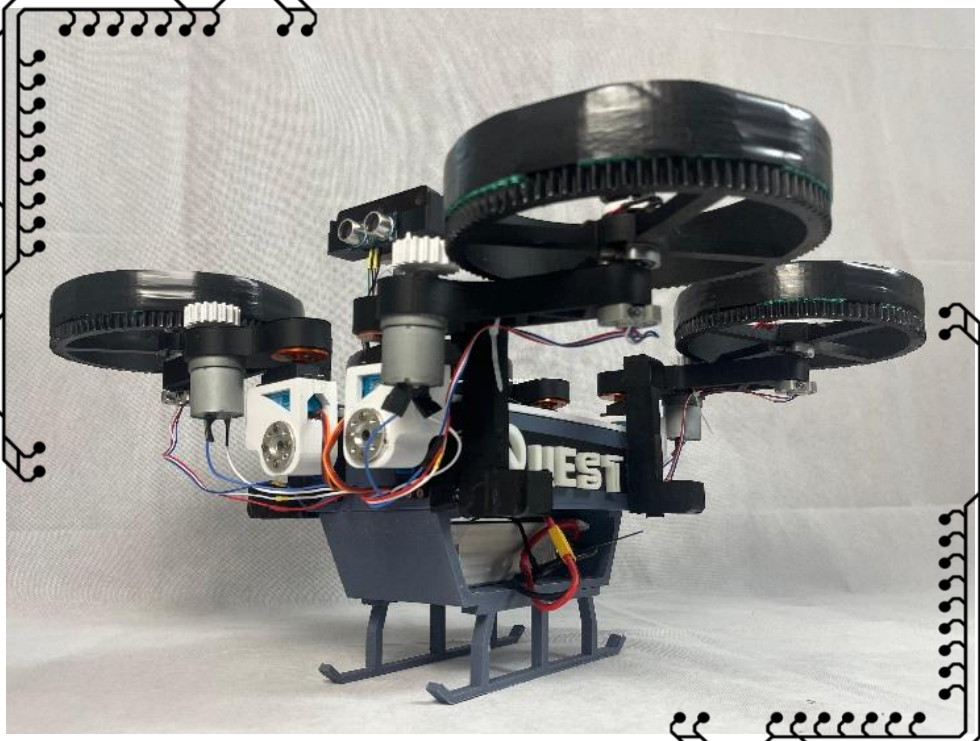
După acestă versiune a urmat o lungă serie de regândire a brațelor si carcaselor. Timp de mai multe zile, pe lângă design-ul componentelor, au fost realizate lipiturile de fire între componentele principale ale scheletului.





Ulterior, trecerea de la capurile de servo de plastic, la cele metalice, și folosirea unui sistem cu magneți, ne-a ajutat să stabilizăm bine robotul fără mari diferențe de greutate. Totodată, partea software a avansat în corcordanță cu situația robotului și a componentelor electronice. Astfel nu a durat mult și robotul a început să se miște. Să se miște din puțin, în mai mult, și tot mai mult, până la varianta acutuală.

e. Poze robot



QUEST

QUEST